

Competitive Programming

Section 47: Meet-In-The-Middle

Competitive Programming Course

2026-07-10

Table of contents

1	Split exponential search	3
1.1	Basic pattern	3
1.2	Count subset sums at most X	3
1.3	Best subset sum at most X	3
1.4	Closest subset sum to target	3
1.5	Why this is not just brute force	4
1.6	Common mistakes	4
1.7	Worked split example	4
1.8	Implementation skeleton	4
1.9	Reconstruction note	5
1.10	Memory estimates	5
2	Exercises	6
3	MITM invariant	7

1 Split exponential search

When n is around 30 or 40, 2^n is too large but $2^{(n/2)}$ is fine. Split the set, enumerate both halves, sort one half, and use binary search to combine. This turns many subset problems from impossible to practical.

1.1 Basic pattern

Split the array into L and R.

```
all_sums_L = sums of every subset of L
all_sums_R = sums of every subset of R
```

If $n = 40$, each half has size 20, so each list has about one million sums. That is large but practical in C++ and still manageable in optimized Python for small local limits.

1.2 Count subset sums at most X

Sort `all_sums_R`. For each s in `all_sums_L`, the number of compatible right sums is:

```
upper_bound(all_sums_R, X - s)
```

Add these counts.

1.3 Best subset sum at most X

Again sort right sums. For each left sum s , find the largest right sum $\leq X-s$. Update the best total.

```
candidate = s + right[pos]
```

1.4 Closest subset sum to target

For each left sum s , binary search the right list around $T-s$. Check both the first right sum $\geq T-s$ and the one immediately before it. One of those is closest for that fixed left sum.

1.5 Why this is not just brute force

Meet-in-the-middle still enumerates all subsets, but it avoids enumerating all pairs of subsets. Sorting one side turns combination into logarithmic search.

```
240                impossible
220 + 220 log     practical
```

1.6 Common mistakes

- Forgetting the empty subset.
- Using `int` for sums that require 64-bit.
- Counting subsets when the task asks for distinct sums, or the reverse.
- Sorting both halves when only one sorted half is needed.

1.7 Worked split example

Array:

```
[3, 8, 2, 6], X = 11
```

Split:

```
L = [3,8]
R = [2,6]
```

Subset sums:

```
left:  0, 3, 8, 11
right: 0, 2, 6, 8
```

For left sum 3, the best right sum is 8, giving 11. For left sum 8, the best right sum is 2, giving 10. The best overall is 11.

The full subset behind 11 can be either [3,2,6] or [3,8]; if the problem asks only for the value, no reconstruction is needed.

1.8 Implementation skeleton

Generate subset sums by growing a list:

```
def subset_sums(arr):
    sums = [0]
    for x in arr:
        sums += [s + x for s in sums]
    return sums
```

For counting, use `upper_bound`. For maximizing, use `upper_bound` and step one position back. For closest value, check both neighbors around `lower_bound`.

Proof checklist:

```
Every subset chooses elements from the left half and from the right half.  
These choices are independent.  
Every full subset appears exactly once as left_sum + right_sum.  
Binary search finds the best compatible right sum for a fixed left sum.
```

1.9 Reconstruction note

If a problem asks for the subset itself, store masks with the sums:

```
(sum, mask)
```

After finding the best left and right masks, expand them back into original indices. Keep left and right masks separate because bit 0 in the right half means original index `len(left)`.

For counting, do not store masks; they waste memory.

1.10 Memory estimates

Half size 20 gives about 1,048,576 sums. With 64-bit integers, one list is about 8 MB plus container overhead. Two such lists are fine in C++ and usually fine in Python for local constraints, but masks or extra tuples can multiply memory quickly.

If values are nonnegative and the task is only best sum $\leq X$, you may discard left sums greater than X . If negative values are allowed, that pruning is not valid.

2 Exercises

3 MITM invariant

Split the items into two halves, enumerate every subset sum of each half, and combine only one sum from each list. This is exhaustive because every original subset has exactly one left and one right part. Sort one list; use upper_bound for counts, predecessor search for best feasible sum, and nearest-neighbor checks for closest sum. Complexity is $O(2^{n/2} \log 2^{n/2})$, not $O(2^n)$. Test negative values, empty halves, and a target below every feasible sum.

- A. Count Subset Sums $\leq X$: Count subsets whose sum is at most X .
- B. Best Subset Sum $\leq X$: Print the maximum subset sum not exceeding X .
- C. Closest Subset Sum: Print the minimum absolute difference between a subset sum and target T .